

Einsendeaufgabe Softwaretechnik

ESA Metriken

4. Semester SoSe26

Eingereicht von: Marvin Bock, 106334

Kurs: Softwaretechnik (BHT MIB 20 S26)

Kursleitung: Prof. Dr.-Ing. Stefan Edlich

Für die Einsendeaufgabe habe ich zunächst Checkstyle in Eclipse auf das Projekt ShoppingCart angewendet.

Hier die erste Analyse nach dem Aktivieren von Checkstyle in dem Projekt.

Klasse ShoppingCart:



The screenshot shows the Eclipse IDE with two tabs: *ShoppingCart_Test.java and ShoppingCart.java. The ShoppingCart.java file is open, displaying Java code for a ShoppingCart class. The code is annotated with numerous Checkstyle error markers (red squiggly lines) indicating violations. The code is as follows:

```
1 package shoppingcart;
2
3 import java.util.HashMap;
4
5
6 public class ShoppingCart {
7
8     public Map<String, Integer> warenkorb = new HashMap<>();
9     public Map<String, Integer> warenmenge = new HashMap<>();
10
11     public int getTotal() {
12         int gesamt = 0;
13
14         for (String name: warenkorb.keySet()) {
15             gesamt += warenkorb.get(name) * warenmenge.get(name);
16         }
17         return gesamt;
18     }
19
20     public void addItem(String name, int preis) {
21         if (preis <= 0) {
22             throw new IllegalArgumentException("Der Preis muss größer 0 sein");
23         }
24
25         warenkorb.put(name, preis);
26
27         if (warenmenge.containsKey(name)) {
28             warenmenge.put(name, warenmenge.get(name) + 1);
29         } else {
30             warenmenge.put(name, 1);
31         }
32     }
33
34     public int getQuantity(String name) {
35         return warenmenge.getOrDefault(name, 0);
36     }
37
38     public void removeItem(String name) {
39
40         int menge = warenmenge.get(name);
41
42         if (menge > 1) {
43             warenmenge.put(name, menge - 1);
44         } else {
45             warenmenge.remove(name);
46             warenkorb.remove(name);
47         }
48     }
49 }
50
51
52 }
53
```

Die häufigste Warnung lautetet „Line has trailing spaces“, mit Hilfe von STRG + SHIFT + F wird in Eclipse das Format geprüft und überflüssig Tabs oder Leerzeichen entfernt.

Hier das Ergebnis:

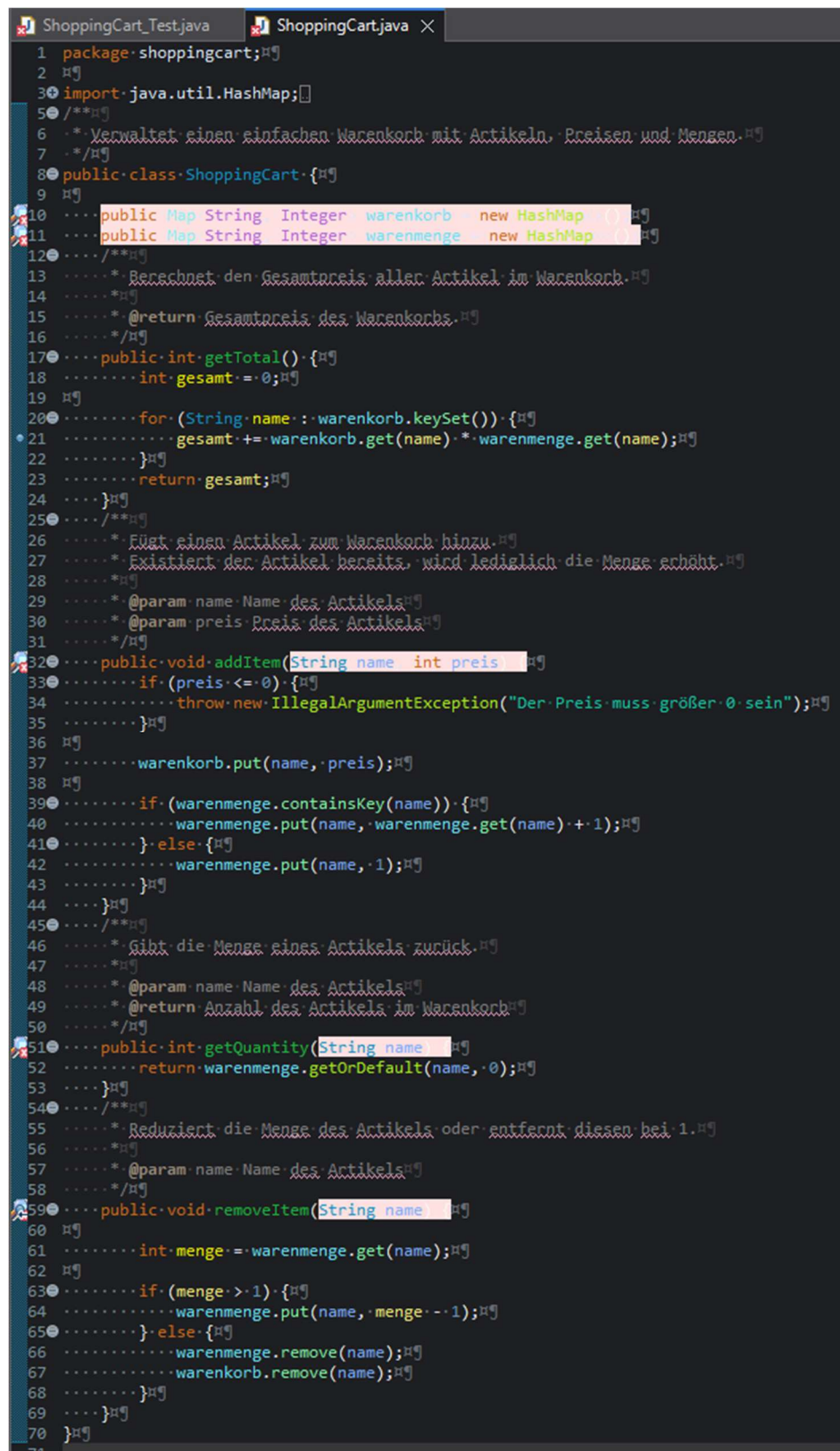


```
1 package shoppingcart;
2
3 import java.util.HashMap;
4
5
6 public class ShoppingCart {
7
8     public Map<String, Integer> warenkorb = new HashMap<>();
9     public Map<String, Integer> warenmenge = new HashMap<>();
10
11     public int getTotal() {
12         int gesamt = 0;
13
14         for (String name : warenkorb.keySet()) {
15             gesamt += warenkorb.get(name) * warenmenge.get(name);
16         }
17         return gesamt;
18     }
19
20     public void addItem(String name, int preis) {
21         if (preis <= 0) {
22             throw new IllegalArgumentException("Der Preis muss größer 0 sein");
23         }
24
25         warenkorb.put(name, preis);
26
27         if (warenmenge.containsKey(name)) {
28             warenmenge.put(name, warenmenge.get(name) + 1);
29         } else {
30             warenmenge.put(name, 1);
31         }
32     }
33
34     public int getQuantity(String name) {
35         return warenmenge.getOrDefault(name, 0);
36     }
37
38     public void removeItem(String name) {
39
40         int menge = warenmenge.get(name);
41
42         if (menge > 1) {
43             warenmenge.put(name, menge - 1);
44         } else {
45             warenmenge.remove(name);
46             warenkorb.remove(name);
47         }
48     }
49 }
```

Da nicht alle Warnungen behoben waren, habe ich mir die Leerzeichen und Tabs anzeigen lassen, anschließend den Formatter von Tabs only auf Spaces only umgestellt und erneut STRG+SHIFT+F angewendet. Nun wurden alle Tabs in Spaces umgewandelt. Ich habe noch einige Kommentare ergänzt. Die letzten Warnungen sind, dass die

HashMap nicht Private ist und dass die Parameter in den Methoden nicht final sind. Damit die Testklasse noch funktioniert habe ich dies unverändert gelassen.

Hier das Finale Ergebnis:

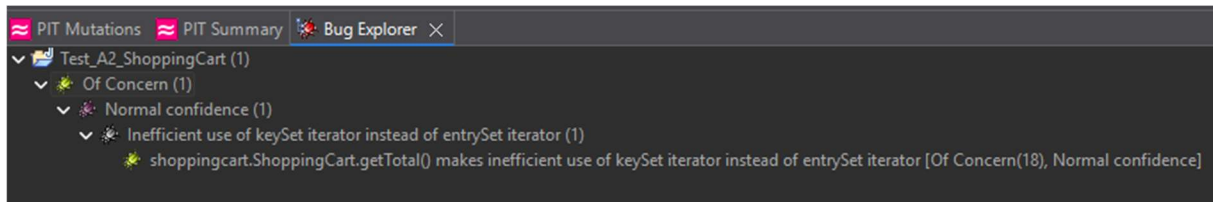


```
1 package shoppingcart;
2
3 import java.util.HashMap;
4
5 /**
6  * Verwaltet einen einfachen Warenkorb mit Artikeln, Preisen und Mengen.
7  */
8 public class ShoppingCart {
9
10     public Map<String, Integer> warenkorb = new HashMap<>();
11     public Map<String, Integer> warenmenge = new HashMap<>();
12
13     /**
14      * Berechnet den Gesamtpreis aller Artikel im Warenkorb.
15      *
16      * @return Gesamtpreis des Warenkorbs.
17      */
18     public int getTotal() {
19         int gesamt = 0;
20         for (String name : warenkorb.keySet()) {
21             gesamt += warenkorb.get(name) * warenmenge.get(name);
22         }
23         return gesamt;
24     }
25
26     /**
27      * Fügt einen Artikel zum Warenkorb hinzu.
28      * Existiert der Artikel bereits, wird lediglich die Menge erhöht.
29      *
30      * @param name Name des Artikels
31      * @param preis Preis des Artikels
32      */
33     public void addItem(String name, int preis) {
34         if (preis <= 0) {
35             throw new IllegalArgumentException("Der Preis muss größer 0 sein");
36         }
37         warenkorb.put(name, preis);
38
39         if (warenmenge.containsKey(name)) {
40             warenmenge.put(name, warenmenge.get(name) + 1);
41         } else {
42             warenmenge.put(name, 1);
43         }
44     }
45
46     /**
47      * Gibt die Menge eines Artikels zurück.
48      *
49      * @param name Name des Artikels
50      * @return Anzahl des Artikels im Warenkorb
51      */
52     public int getQuantity(String name) {
53         return warenmenge.getOrDefault(name, 0);
54     }
55
56     /**
57      * Reduziert die Menge des Artikels oder entfernt diesen bei 1.
58      *
59      * @param name Name des Artikels
60      */
61     public void removeItem(String name) {
62         int menge = warenmenge.get(name);
63         if (menge > 1) {
64             warenmenge.put(name, menge - 1);
65         } else {
66             warenmenge.remove(name);
67             warenkorb.remove(name);
68         }
69     }
70 }
71
```

Es scheint mir ein sehr hilfreiches und cooles Plugin zu sein.

Als zweites Werkzeug habe ich versucht PMD in Eclipse einzurichten. Leider war dies so einfach nicht möglich, es fehlte ein Ruleset, weshalb ich es abgebrochen habe. Leider habe ich bei SpotBugs nicht direkt gesehen, dass das Ergebnis in einem eigenen Explorer angezeigt wird und nicht direkt nach der Analyse eingeblendet wird. Auch hier vermutete ich erst eine fehlerhafte Installation.

Hier nun das Ergebnis der Analyse:



Der betroffene Code Abschnitt lautet:

```
for (String name : warenkorb.keySet()) {  
    gesamt += warenkorb.get(name) * warenmenge.get(name);  
}
```

In meiner getTotal()-Methode nutze ich die keySet()-Schleife nicht ganz optimal. Ich gehe dort über alle Schlüssel der Map und hole mir dann für jeden Schlüssel nochmal den Wert mit get().

SpotBugs sagt, dass das ineffizient ist und ich stattdessen entrySet() benutzen soll, weil ich dann Schlüssel und Wert direkt zusammen habe und nicht doppelt auf die Map zugreifen muss.

Bei großen Maps wäre dies ein großer Vorteil.